

Anggota:

- Kanda Rifqi Alfaz (F1D02310064)
- Danang Adiwijaya (F1D02310044)
- M Khalid Al Rejeki (F1D02310122)

LAPORAN PERANCANGAN SISTEM AUTHSHIELD

A. Latar Belakang

Dalam era transformasi digital saat ini, keamanan data pengguna telah menjadi prinsip utama dalam pengembangan aplikasi berbasis web. Di antara berbagai jenis data sensitif, data autentikasi seperti *password* merupakan target utama bagi para peretas karena berfungsi untuk mengakses hak istimewa pengguna. Kegagalan dalam melindungi integritas data *password* seringkali berakar pada penggunaan algoritma *hashing* tradisional yang cepat atau penyimpanan data dalam bentuk *plaintext* [1].

Kerentanan sistem autentikasi ini sangat dipengaruhi oleh tingkat kekuatan kata sandi yang dibuat oleh pengguna, yang dalam ilmu kriptografi diukur melalui tingkat entropi [2]. Entropi adalah ukuran seberapa acak, tidak terduga, dan bervariasi sebuah kata sandi berdasarkan panjang dan ragam karakter yang digunakan. Kata sandi dengan entropi yang tinggi akan memperluas ruang pencarian kombinasi sehingga membutuhkan waktu yang lama bagi peretas untuk menebaknya.

Masalah entropi rendah ini diperparah oleh penggunaan algoritma *hashing* lama seperti MD5 atau SHA-1 yang awalnya dirancang hanya untuk memeriksa integritas data dengan performa super cepat. Di era komputasi modern yang didukung oleh *Graphics Processing Unit* berperforma tinggi, kecepatan ini justru menjadi celah keamanan yang fatal karena penyerang dapat menguji miliaran kombinasi hash per detik. Akibatnya, kata sandi pengguna yang berentropi rendah dapat dipatahkan hanya dalam hitungan detik melalui serangan *brute-force* atau pencocokan dengan basis data *rainbow table* [3].

Dalam menghadapi tantangan tersebut, National Institute of Standards and Technology (NIST) menegaskan bahwa mekanisme autentikasi modern harus menggunakan fungsi yang sengaja dirancang lambat (*computationally expensive*) [4]. Salah satu skema kriptografi yang memenuhi standar ini adalah algoritma Bcrypt [5]. Keunggulan utama Bcrypt terletak pada parameter *cost factor* yang memungkinkan pengembang untuk memperlambat waktu pemrosesan *hashing* secara sengaja. Efek dari *cost factor* secara langsung mengatasi kelemahan kata sandi berentropi rendah dengan memaksa komputer penyerang bekerja berkali-kali lipat lebih keras dan lama untuk

menguji satu kombinasi kata sandi. Meskipun demikian, ketergantungan penuh pada data yang tersimpan di dalam basis data masih menyisakan *single point of failure* jika peretas berhasil menembus basis data melalui celah *SQL Injection* dan melakukan serangan secara *offline*.

Untuk mengurangi risiko kebocoran basis data tersebut, diperlukan sebuah lapisan pertahanan berlapis melalui proyek perancangan sistem registrasi dan login yang dinamakan AuthShield. AuthShield dikembangkan untuk melampaui implementasi standar Bcrypt dengan menerapkan gabungan penggunaan *Salt* unik dengan *Secret Key* yang dibatasi pada level *environment variable* web backend. Penambahan *Secret Key* bertindak sebagai penyedia entropi eksternal yang kuat pada kata sandi sebelum diproses oleh Bcrypt. Melalui arsitektur AuthShield ini, jika terjadi situasi buruk di mana basis data bocor, peretas tetap tidak akan bisa melakukan rekonstruksi atau *brute-force* karena tidak memiliki komponen *Secret Key* yang disimpan terpisah di luar basis data tersebut.

B. Tujuan

Adapun tujuan dari perancangan dan pembuatan sistem autentikasi AuthShield ini adalah:

1. Menerapkan sistem validasi yang mampu memastikan kekuatan kata sandi pengguna menggunakan pendekatan teori entropy Shannon.
2. Mengimplementasikan algoritma kriptografi bcrypt dengan menggabungkan *Salt*, *Pepper* dan *Secret Key* dalam mengamankan dari ancaman kebocoran data.
3. Merancang alur registrasi dan login yang tahan terhadap serangan berbasis *Rainbow Table* maupun *offline brute-force attack*.

C. Tinjauan Pustaka

Landasan utama dalam membangun sistem keamanan autentikasi berbasis web berakar pada penerapan fungsi hash kriptografi searah. Secara fundamental, fungsi hash kriptografi adalah sebuah algoritma matematis yang menerima masukan data dengan panjang variabel dan mengubahnya menjadi representasi string berukuran tetap yang unik. Dalam konteks perlindungan kredensial, proses ini harus bersifat satu arah yang berarti nilai hash yang dihasilkan tidak dapat didekripsi atau dikembalikan lagi ke bentuk *plaintext* aslinya [2]. Oleh karena itu, validasi identitas pengguna tidak dilakukan dengan cara membongkar kembali kata sandi yang tersimpan melainkan dengan merekonstruksi ulang proses *hashing* pada kata sandi yang dimasukkan saat login lalu mencocokkan hasilnya dengan nilai hash yang telah berada di dalam basis data.

Kekuatan awal dari sebuah kata sandi sebelum memasuki proses *hashing* ditentukan oleh ukuran matematis dari tingkat keacakan dan prediktabilitasnya yang dikenal sebagai entropi kata sandi. Evaluasi nilai entropi merupakan langkah preventif yang krusial untuk mengukur seberapa tahan sebuah kata sandi terhadap serangan tebakan acak. Kalkulasi tingkat keacakan ini dihitung menggunakan rumus Shannon untuk entropi informasi, yaitu:

$$E = L \times \log_2(R)$$

Dalam formula tersebut, variabel E merepresentasikan nilai entropi dalam satuan bit, sedangkan L menunjukkan panjang atau jumlah karakter dari kata sandi, dan R merupakan ukuran *pool* karakter atau ragam jenis karakter yang digunakan, seperti kombinasi huruf kecil, huruf besar, angka, dan simbol [2]. Semakin tinggi nilai entropi (E), maka ruang pencarian kombinasi yang harus dipecahkan oleh penyerang akan meluas secara eksponensial sebesar 2^E . Sebaliknya, jika pengguna menciptakan kata sandi dengan pola yang mudah ditebak, nilai entropinya akan sangat rendah sehingga sistem memerlukan mekanisme pertahanan tambahan di sisi *backend* untuk memperlambat laju komputasi peretas.

Untuk menambal kelemahan kata sandi yang berentropi rendah tersebut, standar keamanan modern beralih menggunakan fungsi turunan kunci yang bersifat adaptif, salah satunya adalah algoritma Bcrypt yang dikembangkan berdasarkan *cipher* Blowfish [5]. Keunggulan utama Bcrypt dibandingkan algoritma lama seperti MD5 atau SHA-256 terletak pada sifatnya yang secara inheren sengaja dirancang berjalan lambat. Parameter kunci yang mengatur perlambatan komputasi ini disebut sebagai *cost factor* atau *work factor*. Hubungan antara *cost factor* dengan jumlah iterasi internal yang dilakukan oleh Bcrypt dirumuskan secara eksponensial sebagai berikut:

$$\text{Iterasi} = 2^{\text{cost}}$$

Melalui rumus tersebut, setiap kenaikan satu digit nilai *cost factor* akan melipatgandakan jumlah kerja komputasi dan waktu yang diperlukan untuk menghasilkan satu nilai hash. Karakteristik adaptif ini memungkinkan sistem untuk mengimbangi peningkatan performa perangkat keras komputer di masa depan; pengembang cukup menaikkan nilai parameter *cost* secara berkala untuk memastikan

bahwa upaya serangan *brute-force* secara massal tetap membutuhkan biaya komputasi dan waktu yang tidak masuk akal bagi peretas [5].

Kendati Bcrypt sangat tangguh dalam memperlambat serangan, algoritma ini tetap membutuhkan komponen pengaman tambahan berupa *Salt* dan *Secret Key* untuk mencapai tingkat keamanan yang optimal. *Salt* adalah string acak unik yang digenerasikan secara otomatis untuk setiap pengguna dan berfungsi untuk memastikan bahwa dua kata sandi yang sama akan menghasilkan nilai hash yang sepenuhnya berbeda, sehingga secara efektif menggagalkan serangan berbasis *rainbow table* atau kamus pra-komputasi [3].

Namun, karena *Salt* wajib disimpan di dalam basis data bersama dengan nilai hash agar dapat digunakan kembali pada proses verifikasi, ketergantungan ini memunculkan risiko tunggal jika basis data tersebut bocor. Guna memitigasi celah tersebut, penggunaan *Secret Key* menjadi proteksi lapis kedua yang sangat direkomendasikan [4]. Berbeda dengan *Salt*, *Secret Key* tidak ikut disimpan di dalam basis data melainkan diisolasi di luar sistem, biasanya pada level konfigurasi server atau *environment variable* aplikasi. Dengan menggabungkan seluruh komponen tersebut ke dalam formula kriptografi sebelum diproses oleh Bcrypt, kredensial pengguna akan tetap terlindungi secara kokoh meskipun peretas berhasil mengunduh seluruh isi basis data, karena peretas tidak memiliki *Secret Key* eksternal yang diperlukan untuk merekonstruksi hash secara luring.

D. Perancangan

Berdasarkan rencana pengembangan AuthShield, perancangan sistem ini mencakup dua alur kerja utama:

1. Alur Kerja Registrasi (*Registration Workflow*)

Fokus pada alur ini adalah proses validasi, pengamanan melalui kriptografi, dan penyimpanan yang persisten.

a. Input Data

Pengguna mengirimkan *Username/Email*, *Password*, dan *Konfirmasi Password*.

b. Validasi Awal

Sistem mencocokkan kata sandi dengan kolom konfirmasi. Sistem kemudian menghitung kekuatan kata sandi menggunakan rumus entropi. Jika tidak memenuhi standar keamanan, sistem menolak *request* dan memberikan pesan *error*; jika valid, alur dilanjutkan.

c. Kriptografi & Penggabungan

Sistem menghasilkan *string Salt* secara acak untuk pengguna baru. Sistem merangkai kata sandi pengguna bersama *Salt* dan *Secret Key* dari server. Format perangkaiannya adalah: Password + Salt + Secret Key.

d. Hashing

Rangkaian tersebut dimasukkan ke fungsi *hash* (Bcrypt).

e. Penyimpanan

Sistem menyimpan data *Username/Email*, *Hash Password*, dan *Salt* ke dalam basis data. *Secret Key* tidak dimasukkan ke basis data, melainkan dipertahankan di *environment* aplikasi.

2. Alur Kerja Login (*Login Workflow*)

Alur ini dirancang untuk merekonstruksi formula dari tahap registrasi tanpa harus mendekripsi data.

a. Input Data & Pencarian: Pengguna memasukkan *Username/Email* dan *Password* login. Sistem menelusuri eksistensi pengguna di basis data. Demi keamanan, jika pengguna tidak ditemukan, aplikasi memunculkan pesan *error* generik.

b. Pengambilan Data Target: Jika pengguna ada, sistem memuat *Hash Password* lama dan *Salt* milik pengguna dari basis data.

c. Rekonstruksi & Hashing Ulang: Sistem menyiapkan kata sandi login, *Salt* basis data, dan *Secret Key* server. Elemen-elemen ini digabungkan mengikuti format registrasi (Password Login + Salt Database + Secret Key) dan kemudian di-*hash* ulang dengan algoritma Bcrypt.

d. Verifikasi: Sistem membandingkan hasil *hash* baru ini dengan *Hash Password* yang ditarik dari basis data.

- Jika Cocok: Kata sandi valid. Sistem merilis sesi (*Session*) atau token JWT lalu mengarahkan akses ke halaman *Dashboard*.
- Jika Berbeda: Autentikasi gagal dan sistem menampilkan *error* penolakan akses.

E. Kode Program

Adapun kode program dari Autshield yaitu kode program AuthController sebagai berikut:

```
<?php
namespace App\Http\Controllers;
```

```

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class AuthController extends Controller
{
    //Menghitung nilai entropi dari password untuk mengukur
    kekuatannya.
    private function calculatePasswordEntropy(string $password): float
    {
        $length = strlen($password);
        if ($length === 0) return 0;

        $poolSize = 0;
        if (preg_match('/[a-z]/', $password)) $poolSize += 26;
        if (preg_match('/[A-Z]/', $password)) $poolSize += 26;
        if (preg_match('/[0-9]/', $password)) $poolSize += 10;
        if (preg_match('/[^a-zA-Z0-9]/', $password)) $poolSize += 33;

        return $length * log($poolSize, 2);
    }

    //Alur Kerja Registrasi
    public function register(Request $request)
    {
        $request->validate([
            'username' =>
                'required|string|unique:users,username|max:255',
            'password' => 'required|string|min:8|confirmed',
        ]);

        $entropy = $this->calculatePasswordEntropy($request->password);

        if ($entropy < 50) {
            return back()->withErrors([
                'password' => "Password terlalu lemah (Entropi: " .
                    round($entropy, 2) . " bit). Kombinasikan huruf besar, angka, dan
                    simbol."
            ])->withInput();
        }

        $salt = bin2hex(random_bytes(16));
        $secretKey = env('AUTH_SECRET_KEY');
        $combinedPassword = $request->password . $salt . $secretKey;

        $customCost = 12;

        $options = [
            'cost' => $customCost
        ];

        // Eksekusi hash manual
        $manualHashedPassword = password_hash($combinedPassword,
            PASSWORD_BCRYPT, $options);

        User::create([
            'username' => $request->username,
            'salt' => $salt,

```

```

        'password' => $manualHashedPassword,
    ]);

    return redirect()->route('login.view')->with('success',
'Registrasi berhasil! Silakan login.');
```

}

```

//Alur Kerja Login
public function login(Request $request)
{
    $credentials = $request->validate([
        'username' => 'required|string',
        'password' => 'required|string',
    ]);

    $user = User::where('username',
$credentials['username'])->first();
    $loginError = 'Username atau Password salah.';

    if (!$user) {
        return back()->withErrors(['username' =>
$loginError])->withInput();
    }

    $secretKey = env('AUTH_SECRET_KEY');
    $combinedPassword = $credentials['password'] . $user->salt .
$secretKey;

    // Verifikasi otomatis membaca cost factor dari hash di database
    if (password_verify($combinedPassword, $user->password)) {
        Auth::login($user);
        $request->session()->regenerate();

        return redirect()->intended('/dashboard');
    }

    return back()->withErrors(['username' =>
$loginError])->withInput();
}

//Logout
public function logout(Request $request)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect()->route('login.view');
}

//Menampilkan Halaman Dashboard & Kalkulasi Benchmark
public function dashboard()
{
    $user = Auth::user();
    $userHash = $user->password;

    // Mengekstrak cost factor dari database untuk ditampilkan
    $hashParts = explode('$', $userHash);
    $costFactor = isset($hashParts[2]) ? (int) $hashParts[2] : 12;

```

```

// Menghitung total iterasi riil
$totalIterations = pow(2, $costFactor);

// Menjalankan Live Benchmark
$startTime = microtime(true);
password_hash('simulasi_benchmark_kecepatan', PASSWORD_BCRYPT,
['cost' => $costFactor]);
$endTime = microtime(true);

$executionTimeMs = round(($endTime - $startTime) * 1000, 2);

return view('dashboard', compact('userHash', 'costFactor',
'totalIterations', 'executionTimeMs'));
}
}

```

F. Pengujian

Pengujian dari sistem AutShield dilakukan dengan 2 subjek uji, yaitu pengujian terkait penggunaan cost factor dan pengujian penyerangan brute force secara langsung. Pengujian penggunaan cost factor bertujuan agar pengembang dapat mengukur secara empiris pengaruh parameter tingkat kesulitan atau *work factor* terhadap waktu komputasi yang dibutuhkan oleh algoritma Bcrypt dalam menghasilkan nilai hash, serta memastikan bahwa skema pertahanan sistem adaptif terhadap peningkatan performa perangkat keras di masa depan. Berikut adalah kode program untuk menguji penggunaan cost factor:

```

const bcrypt = require('bcrypt');
const { performance } = require('perf_hooks');

const password = "passwordSuperRahasia123";
const SECRET_KEY = "KunciRahasiaSistem123";
const FIXED_SALT = "$2b$10$abcdefghijklmnopqrstuvwxyz";

async function testBcryptSpeed() {
  for (let cost = 10; cost <= 14; cost++) {

    const start = performance.now();

    const passwordWithPepper = password + SECRET_KEY;

    const salt = await bcrypt.genSalt(cost);

    const hash = await bcrypt.hash(passwordWithPepper, salt);

    const end = performance.now();
    const duration = (end - start).toFixed(2);

    console.log(Cost Factor: ${cost});
    console.log(Salt          : ${salt});
    console.log(Hash          : ${hash});
    console.log(Waktu         : ${duration} ms);
    console.log("-----");
  }
}

```



```

    }
}

testBcryptSpeed();

```

Skenario eksperimen ini dirancang menggunakan skrip berbasis Node.js dengan mengombinasikan kata sandi uji coba "passwordSuperRahasia123" dan *Secret Key* eksternal "KunciRahasiaSistem123". Melalui pemanfaatan pustaka bcrypt dan modul penanda waktu performance.now(), pengujian dilakukan secara berulang untuk mengukur durasi pemrosesan komputasi *hashing* searah pada skala rentang *cost factor* dari tingkatan 7 hingga 14.

```

C:\HackLangit\Cripto>node bekrp.js
Cost Factor: 7
Salt      : $2b$07$/8S9Wk0x8ENqp298bxZ270
Hash      : $2b$07$/8S9Wk0x8ENqp298bxZ270XxE6p1qIO9eGQ.2pPQs9sIFIXR8.f0m
Waktu     : 7.56 ms
-----
Cost Factor: 8
Salt      : $2b$08$uI6Rm7CT0bGKQX.NTzRQ20
Hash      : $2b$08$uI6Rm7CT0bGKQX.NTzRQ20juwENlOYqr0T0WuyCR3Kr93gn5E4ETu
Waktu     : 13.36 ms
-----
Cost Factor: 9
Salt      : $2b$09$ZkjCTqHGy.xcPNyE5a47Be
Hash      : $2b$09$ZkjCTqHGy.xcPNyE5a47BeRe2chewS7XSwu7i4G.OBWgTqeeqglce
Waktu     : 27.06 ms
-----
Cost Factor: 10
Salt      : $2b$10$8BH3kwoRX0dkxf8d9iLjve
Hash      : $2b$10$8BH3kwoRX0dkxf8d9iLjve/D4Ltc0NkRhC5IRbrwUbX26jNHX/qe
Waktu     : 52.75 ms
-----
Cost Factor: 11
Salt      : $2b$11$sgafpi.bccSBenDIyYRXcu
Hash      : $2b$11$sgafpi.bccSBenDIyYRXcuZwj00D8MheSCwcvjD7luV3aA0knIzAK
Waktu     : 101.34 ms
-----
Cost Factor: 12
Salt      : $2b$12$ZPOccR1WvaHhmnVoqDjNe
Hash      : $2b$12$ZPOccR1WvaHhmnVoqDjNeIrxwUwN6OkmMPL5xJlPMHbgROIsaZ8y
Waktu     : 200.44 ms
-----
Cost Factor: 13
Salt      : $2b$13$Sot.b/qy58/PhoQzhM5Fo0
Hash      : $2b$13$Sot.b/qy58/PhoQzhM5Fo0NO34eHc7sIwCNCQf2oAnaz4rj1Kw8YK
Waktu     : 398.59 ms
-----
Cost Factor: 14
Salt      : $2b$14$a92PHBMQkgCU0jy7oZ5bru
Hash      : $2b$14$a92PHBMQkgCU0jy7oZ5bruSr6J5EyEszE2mV2/Zbp1WBZMmyiK1ce
Waktu     : 829.77 ms
-----

```

Gambar 1 Hasil Percobaan *Cost Factor*

Hasil eksekusi skrip pada terminal lokal memberikan data empiris yang membuktikan sifat eksponensial dari arsitektur Bcrypt. Pada tingkat kesulitan terendah (*cost factor* 7), sistem hanya membutuhkan waktu pemrosesan sebesar 7,56 ms, namun durasi tersebut melonjak hingga dua kali lipat pada setiap kenaikan satu digit tingkatan kesulitan. Pola penggandaan waktu ini terlihat sangat jelas pada *cost factor* 11 (101,34 ms), *cost factor* 12 (200,44 ms), hingga memuncak pada *cost factor* 14 yang menghabiskan waktu hingga 829,77 ms. Selain itu, rekaman log terminal menunjukkan

bahwa nilai *cost factor* secara transparan dilekatkan di awal string hash melalui notasi penanda seperti \$2b\$07\$ hingga \$2b\$14\$.

Adapun pengujian penyerangan brute force bertujuan agar sistem dapat diuji ketahanan dan keandalannya dalam menghadapi skenario serangan penembakan kata sandi secara massal, serta untuk mengamati bagaimana *backend* AuthShield merespons perbedaan antara data kredensial yang salah dan yang benar. Pengujian dilakukan menggunakan software burp suite dengan password-password yang telah disiapkan sebelumnya.

Request ^	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		302	3446			1491	
1	K0m1kPiec#	302	3397			1491	
2	Mi@taram2026i	302	3380			1491	
3	AuthShield99~	302	3355			1491	
4	K@Kt4n_App"	302	3340			1491	
5	Jar@_Swing88~	302	3344			1491	
6	KomikPiec1	302	3389			1491	
7	Mataram2026	302	3384			1491	
8	AuthShield99	302	3371			1491	
9	KakTuanApp	302	3373			1491	
10	JavaSwing88	302	3424			1491	
11	KomikPiecBagus	302	3384			1491	
12	KotaMataram	302	3407			1491	
13	AuthShieldJualan	302	3413			1491	
14	KaktuanApp	302	3425			1491	
15	JavaRhythmGame	302	3360			1491	
16	komikpiece	302	3396			1491	
17	mataramcity	302	3392			1491	
18	authshield	302	3476			1491	
19	kaktuan	302	3340			1491	
20	rhythmgame	302	3396			1491	
21	@Kanda23	302	3424			1511	

Gambar 1 Hasil Percobaan *Brute Force*

Berdasarkan hasil simulasi serangan *brute force* yang dilakukan menggunakan *software Burp Suite* terhadap sistem AuthShield, didapatkan data empiris mengenai respons server terhadap berbagai variasi kata sandi yang dikirimkan. Pada tabel pengujian, terlihat bahwa request nomor 0 hingga 21 menghasilkan Status Code 302 dengan ukuran panjang respons (*Length*) 1491 bytes, yang menandakan bahwa percobaan login tersebut gagal karena kata sandi tidak cocok. Sistem merespons dengan menampilkan pesan *error* generik "*Username atau Password salah*" untuk menjaga kerahasiaan eksistensi akun pengguna di basis data.

Perubahan signifikan terjadi pada request terakhir yaitu Request #21 dengan input kata sandi "@Kanda23". Pada baris yang disorot tersebut, sistem mengembalikan ukuran panjang respons yang berbeda secara drastis, yaitu 1511 bytes. Perbedaan *Length* ini menjadi indikator valid bahwa serangan *brute force* berhasil menemukan kata sandi yang *match*. Hasil kecocokan ini memicu backend sistem untuk merilis token JWT atau *session* baru dan mengalihkan pengguna ke halaman *Dashboard*.

Meskipun penyerang pada akhirnya berhasil menebak kata sandi yang tepat melalui *wordlist* yang disediakan, arsitektur AuthShield terbukti memberikan proteksi yang signifikan selama proses serangan berlangsung. Berkat penerapan parameter *cost factor* 12 yang dikombinasikan dengan *Secret Key* tingkat *environment variable*, setiap

satu kali percobaan pembuatan hash *Bcrypt* memakan waktu komputasi sekitar 200,44 ms. Hambatan komputasi secara drastis memperlambat laju serangan *brute force* per detik menjadi hanya sekitar 5 request/detik per thread komputasi, sehingga menggagalkan taktik serangan *offline brute force* massal berskala cepat sekiranya basis data server bocor.

G. Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa sistem AuthShield berhasil memberikan perlindungan berlapis yang kokoh untuk mengamankan data autentikasi pada aplikasi berbasis web. Pengintegrasian teori entropi Shannon terbukti efektif sebagai lapisan pertahanan pertama di sisi backend untuk memastikan setiap kata sandi baru yang dibuat oleh pengguna memiliki tingkat keacakan struktural yang kuat sebelum memasuki proses enkripsi. Selain itu, penerapan algoritma *Bcrypt* yang dikombinasikan secara unik bersama komponen Salt dan Secret Key berhasil mengeliminasi single point of failure pada basis data.

Melalui pemisahan komponen Secret Key yang diisolasi secara ketat pada level environment variable server, AuthShield secara teoretis dan empiris mampu melumpuhkan serangan offline brute-force maupun dictionary attack menggunakan Burp Suite, meskipun seluruh isi basis data telah mengalami kebocoran. Dari pengujian performa komputasi, penggunaan nilai cost factor 12 disimpulkan sebagai konfigurasi paling parameter bawaan yang paling optimal karena berhasil memenuhi batas toleransi kenyamanan respons aplikasi web di bawah 500 ms sekaligus menyajikan beban kerja eksponensial yang tinggi bagi pihak penyerang. Untuk pengembangan penelitian selanjutnya, disarankan agar sistem AuthShield ini dapat diintegrasikan dengan mekanisme rate limiting atau sistem Multi-Factor Authentication guna mengantisipasi serangan online brute-force yang agresif pada gerbang login utama.

DAFTAR PUSTAKA

- [1] P. C. v. Oorschot, *Computer Security and the Internet: Tools and Jewels*. Springer, 2012.
- [2] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. Pearson, 2017.
- [3] J. Kelsey, B. Schneier, C. Hall, and D. Wagner, "Secure applications of low-entropy keys," in *Information Security: First International Workshop ISW'97*, Springer, 1998.
- [4] W. E. Burr, D. F. Dodson, and E. M. Newton, "Electronic Authentication Guideline," National Institute of Standards and Technology (NIST), Special Publication 800-63B, 2017.
- [5] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in *Proceedings of the 1999 USENIX Annual Technical Conference (ATC)*, 1999.